

14 января 2026 г. · Исследование

Масштабирование автономного программирования



Уилсон Лин · 7 мин на чтение

Оглавление

Ограничения одного агента
Обучение координации
Планировщики и исполнители
Работа в течение нескольких недель
Что мы узнали
Что дальше



Мы уже несколько недель экспериментируем с автономным запуском агентов для написания кода.

Наша цель — понять, как далеко мы можем продвинуться в области агентного программирования для проектов, на выполнение которых у людей обычно уходят месяцы.

В этом посте мы расскажем о том, что узнали, запустив сотни параллельных агентов для одного проекта, координируя их работу и наблюдая за тем, как они пишут более миллиона строк кода и триллионы токенов.

Ограничения одного агента

Современные агенты хорошо справляются с узконаправленными задачами, но медленно работают над сложными проектами. Естественный следующий шаг — запустить несколько агентов параллельно, но понять, как их координировать, непросто.

Сначала мы подумали, что заблаговременное планирование будет слишком жестким. Путь к реализации крупного проекта неоднозначен, и правильное распределение работы с самого начала неочевидно. Мы начали с динамической координации, при которой агенты решают, что делать, исходя из того, что делают другие.

координировать свои действия через общий файл. Каждый агент проверял, что делают другие, запрашивал задачу и обновлял ее статус. Чтобы два агента не могли взять одну и ту же задачу, мы использовали механизм блокировки.

Он давал интересные сбои:

1. Агенты удерживали блокировки слишком долго или вовсе забывали их снять. Даже когда блокировка работала корректно, она становилась узким местом. Двадцать агентов снизят эффективную пропускную способность до двух-трех, при этом большую часть времени они будут проводить в ожидании.
2. Система была ненадежной: агенты могли выйти из строя, удерживая блокировки, пытаться получить блокировки, которые они уже удерживали, или обновлять координационный файл, вообще не удерживая блокировку.

Мы попробовали заменить блокировки оптимистичным контролем параллелизма. Агенты могли свободно читать состояние, но запись не выполнялась, если состояние изменилось с момента последнего чтения. Это было проще и надежнее, но оставались и более серьезные проблемы.

Без иерархии агенты стали избегать рисков. Они уклонялись от сложных задач и вносили небольшие безопасные изменения. Ни один из агентов не брал на себя ответственность за решение сложных проблем или сквозную реализацию. Это приводило к тому, что работа выполнялась в течение длительного времени без какого-либо прогресса.

Планировщики и исполнители

Следующим нашим шагом стало разделение ролей. Вместо плоской структуры, в которой каждый агент делает все, мы создали конвейер с четко распределенными обязанностями.

- **Планировщики** постоянно изучают кодовую базу и создают задачи. Они могут запускать подпланировщиков для решения конкретных задач, что делает планирование параллельным и рекурсивным.
- **Рабочие** берут задачи и полностью сосредотачиваются на их выполнении. Они не координируют свои действия с другими рабочими и не беспокоятся о общей картине. Они просто выполняют поставленную задачу, пока она не будет закончена, а затем отправляют внесенные изменения.

В конце каждого цикла агент-судья решал, продолжать ли работу, после чего следующая итерация начиналась с нуля. Это решило большинство наших проблем с координацией и позволило масштабировать проекты до очень больших размеров, не допуская туннельного зрения ни у одного из агентов.

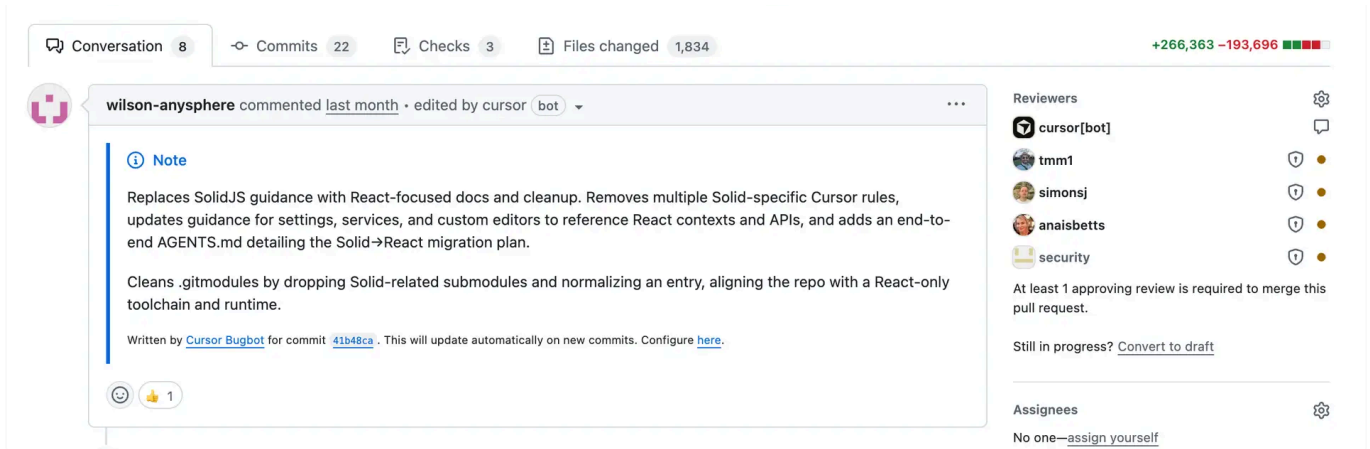
веб-браузер с нуля. Агенты работали почти неделю, написав более 1 миллиона строк кода в 1000 файлах. Вы можете изучить [исходный код на GitHub](#).

Despite the codebase size, new agents can still understand it and make meaningful progress. Hundreds of workers run concurrently, pushing to the same branch with minimal conflicts.



While it might seem like a simple screenshot, building a browser from scratch is extremely difficult.

Another experiment was doing an in-place migration of Solid to React in the Cursor codebase. It took over three weeks with +266K/-193K edits. It still needs careful review, but was passing our CI and early checks.



Another experiment was to improve an upcoming product. A long-running agent made video rendering 25x faster with an efficient Rust version. It also added support to zoom and pan smoothly with natural spring transitions and motion blurs, following the cursor. This code was merged and will be in production soon.

We have a few other interesting examples still running:

- Java LSP: 7.4K commits, 550K LoC
- Windows 7 emulator: 14.6K commits, 1.2M LoC
- Excel: 12K commits, 1.6M LoC

What we've learned

We've deployed trillions of tokens across these agents toward a single goal. The system isn't perfectly efficient, but it's far more effective than we expected.

Model choice matters for extremely long-running tasks. We found that GPT-5.2 models are much better at extended autonomous work: following instructions, keeping focus, avoiding drift, and implementing things precisely and completely.

Opus 4.5 tends to stop earlier and take shortcuts when convenient, yielding back control quickly. We also found that different models excel at different roles. GPT-5.2 is a better planner than GPT-5.1-Codex, even though the latter is trained specifically for coding. We now use the model best suited for each role rather than one universal model.

Many of our improvements came from removing complexity rather than adding it. We initially built an integrator role for quality control and conflict resolution, but found it created more bottlenecks than it solved. Workers were already capable of handling conflicts themselves.

The best system is often simpler than you'd expect. We initially tried to model systems from distributed computing and organizational design. However, not all of them work for agents.

A surprising amount of the system's behavior comes down to how we prompt the agents. Getting them to coordinate well, avoid pathological behaviors, and maintain focus over long periods required extensive experimentation. The harness and models matter, but the prompts matter more.

What's next

Multi-agent coordination remains a hard problem. Our current system works, but we're nowhere near optimal. Planners should wake up when their tasks complete to plan the next step. Agents occasionally run for far too long. We still need periodic fresh starts to combat drift and tunnel vision.

But the core question, can we scale autonomous coding by throwing more agents at a problem, has a more optimistic answer than we expected. Hundreds of agents can work together on a single codebase for weeks, making real progress on ambitious projects.

The techniques we're developing here will eventually inform Cursor's agent capabilities. If you're interested in working on the hardest problems in AI-assisted software development, we'd love to hear from you at hiring@cursor.com. 

Related posts

Filed under: Research

Apr 14, 2026 · Research

Speeding up GPU kernels by 38% with a multi-agent system



Wilson, Sahil, Yuan & Edward · 10 min read

Feb 18, 2026 · Research

Implementing a secure sandbox for local agents



Ani, Yash & Alex · 6 min read

Feb 9, 2026 · Research

Introducing Composer 1.5



Cursor Team · 3 min read

[View more posts →](#)

Product

Agents

Teams

Enterprise

Pricing

Code Review

CLI

Cloud Agents

Composer

Marketplace

Resources

Download

Changelog

Docs

Learn

Value Calculator

Forum

Help

Workshops

Status

Company

Careers

Blog

Community

Students

Brand

Future

Anysphere

Legal

Terms of Service

Privacy Policy

Data Use

Security

Connect

X

LinkedIn

YouTube